Full Stack Developer Machine Test

Technology Stack

- Backend: ASP.NET Core 8 Web API
- Frontend: Angular (v16+ acceptable, v17 preferred)
- Database: SQL Server
- Authentication: JWT
- Source Control: Git

Duration

4–5 Hours

Objective

Develop a simple **Employee Task Management System**.

The application should allow users to manage Employees and their assigned Tasks through a REST API and an Angular frontend.

> **Note on architecture**
> A single well-structured Web API is sufficient and is what will primarily be evaluated. Splitting Employee and Task functionality into two separate microservices is an optional bonus (see Bonus Tasks) — it is **not** required at this level.

Functional Requirements

Employee Module

Employee Fields

- Employee Id
- Employee Code
- FirstName
- LastName
- Email
- Mobile Number
- Department
- Designation
- Date Of Joining
- Is Active
- Created Date

APIs

- Create Employee
- Update Employee
- Delete Employee (Soft Delete)

- Get Employee by Id
- Get All Employees (with Pagination)
- Search Employee by Name/Department

Task Module

Task Fields

- Task Id
- Employee Id
- Title
- Description
- Priority
- Status
- StartDate
- Due Date
- Estimated Hours
- Created Date

APIs

- Create Task
- Update Task
- Delete Task
- Get Task by Id
- Get Tasks by Employee
- Update Task Status

Dashboard API

Create one API that returns:

- Total Employees
- Active Employees
- Pending Tasks
- Completed Tasks
- Overdue Tasks

Authentication

Implement JWT Authentication.

- Login API
- JWT Token Generation
- Only authenticated users should be able to access the Employee/Task/Dashboard APIs

Authentication can use a static/hardcoded user (e.g. one seeded admin user) — a full user management module is not required.

Database

Create SQL tables with:

- Primary Keys
- Foreign Keys
- Default Constraints (e.g. `IsActive`, `CreatedDate`)

Provide the SQL script used to create the tables.

(Indexes are a bonus, not mandatory, at this level.)

Frontend (Angular)

Login Page

- Username, Password
- Store JWT and attach it to subsequent API calls

Dashboard Page

Display cards showing:

- Total Employees
- Active Employees
- Pending Tasks
- Completed Tasks
- Overdue Tasks

Employee Management

- List (with pagination)
- Add / Edit (using two-way data binding)
- Delete
- Search by name or department
- Basic field validation (required fields, valid email)

Task Management

- List tasks with Employee dropdown to assign
- Update task status
- Filter by Status and/or Employee

UI Requirements

- Angular
- Basic form validation with visible error messages
- Loading indicator while an API call is in progress
- Success/error notification after Create/Update/Delete actions

(Full responsive design polish is a bonus, not a requirement — a clean, usable layout is enough.)

Backend Requirements

- ASP.NET Core 8 Web API

- SQL Stored Procedure (SP's)
- Dependency Injection
- Async/Await for all data access
- DTOs for request/response models (don't expose EF entities directly)
- Basic global exception handling (a try/catch middleware or filter is enough)
- Console/file logging for key actions (create, update, delete, login)

(Clean Architecture layering and the Repository Pattern are welcomed if you're comfortable with them, but a simpler 2–3 layer structure — Controller → Service → DbContext — is completely acceptable and will not be penalized.)

SQL Requirements

Write SQL queries for the following:

Query 1

List all employees along with their total number of assigned tasks.

Query 2

Find all overdue tasks (DueDate has passed and Status is not Completed).

Query 3

Find department-wise employee count.

Query 4

Find the top 3 employees with the most completed tasks.

Validation

Employee

- Email must be unique.
- Joining Date cannot be a future date.

Task

- Due Date cannot be earlier than Start Date.
- A Completed task cannot be changed back to Pending.

Bonus Tasks

Implement any **TWO** (optional — do not attempt these before the core requirements above are complete and working):

- Swagger Documentation
- Split Employee and Task into two separate microservices
- Unit Testing (a few tests on the service layer is enough)
- Export Employees to Excel or CSV
- Mobile-responsive UI polish
- Docker support for the API

Deliverables

1. Source Code (Backend + Frontend)
2. SQL Script
3. README.md — how to run the project locally, and any assumptions you made
4. Postman Collection or Swagger link
5. Git Repository Link (or a zipped repo with commit history intact)

Evaluation Criteria

Area	Marks
API Development (CRUD + logic)	25
Database Design & SQL Queries	20
Angular UI & Functionality	20
Authentication (JWT)	10
Validation	10
Code Quality & Structure	10
Git & Documentation	5
Total	**100**

What We're Looking For At This Level

At 2–5 years of experience, we're primarily evaluating whether you can:

- Independently build a working, end-to-end CRUD application without hand-holding
- Write clean, readable code with sensible separation of concerns
- Handle validation and errors gracefully rather than letting the app crash
- Make reasonable, defensible technical decisions on your own (e.g. how to structure a service layer)

We are **not** primarily grading on advanced patterns (CQRS, event-driven design, full microservice orchestration) — those are welcome as bonus effort, but a simple, correct, well-organized solution will score higher than an overengineered one that doesn't fully work.

General Guidelines

- Use meaningful naming conventions.
- Avoid hardcoded values where reasonably possible (e.g. connection strings, JWT secret via config).
- Commit code incrementally to Git rather than one final commit.
- Ensure the application builds and runs with minimal setup — note any manual steps clearly in the README.